

Le chat musicien : un modèle GPT pour la génération musicale

[Mazari Kaïs] — M1 Mathématiques et Applications, Université de Lille
Cours de N. Wicker — Projet IA, sujet n°2

Mai 2026

1 Introduction

Le sujet n°2 du projet demande de construire un LLM qui apprend à prédire de la musique à partir de fichiers MIDI, et d'observer ce que le modèle génère lorsqu'on lui fournit quelques notes en entrée. L'approche que j'ai retenue consiste à transposer l'architecture GPT vue en cours, initialement conçue pour le texte, au domaine de la musique symbolique. Le mécanisme central reste identique : un transformeur décodeur prédit le prochain élément d'une séquence à partir des éléments précédents. La différence se situe au niveau du vocabulaire : au lieu de mots, on manipule des événements musicaux discrets représentant des notes, des positions rythmiques et des nuances.

Ce rapport présente d'abord la chaîne de traitement mise en place pour transformer les fichiers MIDI bruts en séquences de tokens exploitables par un transformeur (section 2). Il décrit ensuite l'architecture du modèle et les choix d'hyperparamètres retenus (section 3), avant de détailler la phase d'entraînement (section 4). La section 5 analyse les cinq pièces générées en faisant varier les paramètres d'échantillonnage. Une discussion des limites observées clôt le rapport.

2 Représentation des données

2.1 Le dataset *GrandMidiPiano*

Le corpus fourni par l'enseignant contient 10 855 fichiers MIDI de piano solo, pour une taille totale de 184 Mo. Avant toute conception du tokenizer, j'ai effectué une exploration statistique sur un échantillon de 2 000 fichiers afin de calibrer mes choix. Trois résultats importants en ressortent : 99% des hauteurs MIDI utilisées sont comprises entre 32 et 95, ce qui couvre largement les 88 touches du piano standard ; la durée médiane d'une note est de 0,65 *beat*, avec un 95^e percentile à 3,85 *beats* ; enfin, tous les fichiers utilisent la même résolution de 384 *ticks* par *beat*, ce qui simplifie le prétraitement.

2.2 Le choix du tokenizer *REMI*

Une approche naïve consisterait à utiliser comme tokens les positions temporelles brutes des notes, exprimées en *ticks*. Cette solution, esquissée dans certains exemples du cours, conduit à un vocabulaire qui croît avec la longueur des morceaux et atteint rapidement plusieurs dizaines de milliers d'entrées différentes. Le modèle peinerait alors à généraliser, et les distributions seraient creuses.

J'ai préféré la représentation *REMI* (*Revamped MIDI*), introduite par Huang et Yang en 2020 pour le *Pop Music Transformer*. L'idée est de quantifier le temps sur une grille rythmique régulière : chaque note est décrite par sa position dans une mesure (en seizièmes de *beat*), sa hauteur, sa durée et sa vélocité. Le vocabulaire devient alors fini et de petite taille. Concrètement, chaque token appartient à l'une des familles suivantes :

- quatre tokens spéciaux : <PAD>, <BOS>, <EOS>, <UNK> ;
- un marqueur de mesure *BAR* (l'ensemble du corpus est en 4/4) ;
- seize tokens de position *POSITION_0* à *POSITION_15*, soit une grille de seizièmes ;

- 88 tokens de hauteur `NOTE_ON_21` à `NOTE_ON_108`, couvrant le piano ;
- seize tokens de durée `DURATION_1` à `DURATION_16` ;
- 32 tokens de vitesse `VELOCITY_0` à `VELOCITY_31`.

Le vocabulaire compte au total 157 tokens. C’est environ 320 fois moins que les 50 257 tokens de GPT-2, ce qui rend l’apprentissage beaucoup plus accessible avec des moyens limités.

2.3 Encodage et statistiques du corpus tokenisé

L’encodage parcourt les notes triées par instant de départ. Pour chaque note, il calcule la mesure et la position correspondantes sur la grille en seizièmes, puis émet la séquence `POSITION ; NOTE_ON ; DURATION ; VELOCITY`. Un nouveau token `BAR` ou `POSITION` n’est inséré que lorsqu’il y a un changement effectif, ce qui permet à plusieurs notes simultanées de partager la même position pour former un accord. Chaque morceau est encadré par `<BOS>` et `<EOS>`.

J’ai exécuté cet encodage en parallèle sur les 10 855 fichiers à l’aide de `ProcessPoolExecutor`. 10 526 fichiers (97%) ont produit une séquence valide, c’est-à-dire dont la longueur est comprise entre 64 et 50 000 tokens. Les morceaux dépassant 16 384 tokens ont été tronqués à cette limite afin d’écartier les valeurs aberrantes sans perdre du contenu utile. Le corpus tokenisé final contient 89,1 millions de tokens, partagés en un ensemble d’entraînement de 10 000 morceaux (170 Mo sérialisés en `pickle`) et un ensemble de validation de 526 morceaux (8,8 Mo).

3 Architecture du modèle

J’ai repris fidèlement l’architecture présentée dans le polycopié de cours (`LLM.pdf`) et dans *Build a Large Language Model from Scratch* de S. Raschka. Le modèle est un transformeur décodeur autorégressif. À l’entrée, les indices des tokens sont projetés par une table d’*embedding* de mots, à laquelle on ajoute un *embedding* positionnel appris. La somme passe ensuite par un *dropout*, puis par une pile de blocs transformeurs identiques. Une normalisation finale et une projection linéaire vers la dimension du vocabulaire produisent les *logits*.

Chaque bloc transformeur applique successivement deux sous-couches encadrées par des connexions résiduelles : une attention multi-tête causale précédée d’un `LayerNorm`, puis un réseau *feed-forward* à expansion 4× avec activation GELU, lui aussi précédé d’un `LayerNorm`. L’attention est masquée par un triangle supérieur de *moins l’infini* pour empêcher chaque position de regarder le futur, conformément à ce qui est attendu d’un modèle autorégressif. Le tableau ci-dessous récapitule les hyperparamètres choisis et leur justification.

Hyperparamètre	Valeur	Justification
<code>vocab_size</code>	157	taille du vocabulaire REMI
<code>context_length</code>	256	environ 16 mesures, suffisant pour la cohérence locale
<code>emb_dim</code>	256	adapté au petit vocabulaire (768 pour GPT-2)
<code>n_heads</code>	8	dimension par tête de 32, équilibre raisonnable
<code>n_layers</code>	6	compromis entre profondeur et temps d’entraînement
<code>drop_rate</code>	0,1	régularisation classique
<code>qkv_bias</code>	False	convention récente, sans impact mesuré sur la perte
Total paramètres	4,88 millions	

Table 1 – Configuration du modèle. Avec un corpus de 89 M de tokens et 4,88 M de paramètres, le ratio est de 18 tokens par paramètre, ce qui correspond à la zone optimale recommandée par Hoffmann *et al.* pour éviter à la fois le sous-entraînement et la sur-paramétrisation.

4 Entraînement

La fonction de perte est l'entropie croisée standard, calculée à chaque position entre la distribution prédite par le modèle et l'indice du token effectivement présent à la position suivante. C'est ce qu'on appelle la prédiction du token suivant (*next-token prediction*), et c'est exactement ce que demande la consigne du projet appliquée au cas musical.

J'utilise un optimiseur **AdamW** avec un taux d'apprentissage maximal de $3 \cdot 10^{-4}$, des paramètres $\beta = (0,9; 0,95)$ et un *weight decay* de 0,01. Le *scheduler* comprend un *warmup* linéaire sur 500 *steps*, suivi d'une décroissance cosinus jusqu'à zéro. Les gradients sont écrêtés à une norme de 1,0 pour stabiliser l'apprentissage. Les fenêtres d'entraînement font 256 tokens avec un pas de 128 (recouvrement de 50%), ce qui produit 661 877 fenêtres d'entraînement et 34 169 fenêtres de validation à partir des 89 M de tokens.

Le développement a été fait en local sous Visual Studio Code, mais l'entraînement a été lancé sur Google Colab afin de bénéficier d'un GPU Tesla T4 (mon ordinateur portable ne dispose que d'un GPU intégré Intel Iris Xe sans CUDA). J'ai prévu des sauvegardes automatiques sur Google Drive toutes les 500 *steps* pour ne rien perdre en cas de déconnexion, ainsi que la possibilité de reprendre à partir du dernier point de sauvegarde grâce à l'option **-resume**. L'entraînement complet de deux *epochs* a duré 139 minutes pour une taille de *batch* de 64.

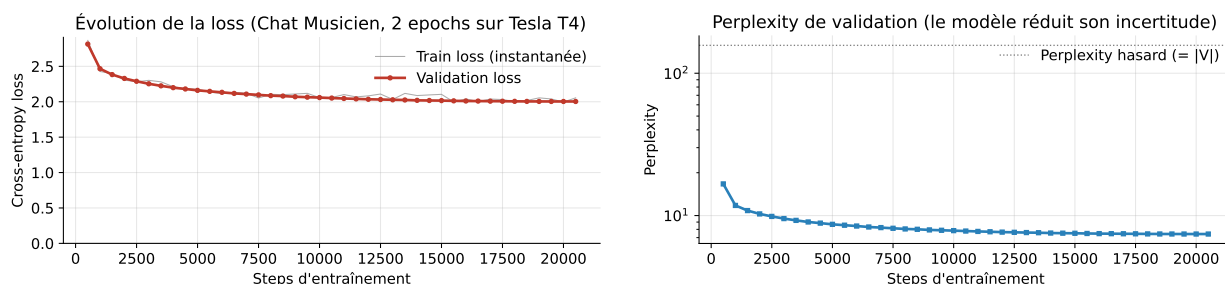


Figure 1 – Évolution de la perte (à gauche, échelle linéaire) et de la perplexité (à droite, échelle logarithmique) au cours de l'entraînement. La perte de validation suit étroitement celle d'entraînement, ce qui indique l'absence de sur-apprentissage. La perplexité passe de la valeur du hasard (157) à 7,42 en fin d'entraînement.

À la fin des deux *epochs*, la perte de validation atteint 2,00 et la perplexité associée vaut 7,42. Concrètement, cela signifie qu'à chaque position, le modèle a réduit son incertitude au point qu'environ sept tokens parmi 157 sont jugés probables, contre 157 dans le cas du hasard uniforme. La courbe de validation reste légèrement au-dessus de celle d'entraînement, mais sans divergence, ce qui exclut un phénomène de sur-apprentissage.

5 Génération et résultats

5.1 Stratégies d'échantillonnage

Une fois le modèle entraîné, la génération se fait de manière autorégressive : à chaque *step*, on calcule les *logits* pour la position suivante, on les transforme en distribution de probabilité par *softmax*, et on échantillonne le prochain token. Le token tiré est ajouté à la séquence et le processus recommence. Trois leviers permettent de contrôler l'équilibre entre fidélité et créativité : la température T , qui aplatit la distribution lorsqu'elle est supérieure à 1 et la concentre lorsqu'elle est inférieure ; le *top-k* qui restreint l'échantillonnage aux k tokens les plus probables ; et le *top-p* (*nucleus*) qui restreint l'échantillonnage à la plus petite enveloppe couvrant une masse de probabilité p .

Choisir simplement l'*argmax* (équivalent à $T \rightarrow 0$) revient à toujours sélectionner le token le plus probable, ce qui produit des séquences extrêmement répétitives et musicalement plates. À l'inverse, échantillonner sans contrainte sur toute la distribution autorise des choix très improbables qui dégradent la cohérence. L'enjeu pratique est de trouver des combinaisons offrant un compromis acceptable.

5.2 Cinq pièces générées

J’ai produit cinq fichiers MIDI couvrant trois axes de variation : le mode de démarrage (à froid à partir de <BOS> ou continuation d’un *prompt* MIDI réel) ; le régime d’échantillonnage (conservateur, équilibré, créatif) ; la longueur (entre 700 et 800 tokens, soit environ 50 secondes à 110 BPM). Le tableau 2 récapitule leurs caractéristiques. Les pièces sont fournies dans le dossier `outputs/generated/` de l’archive de code.

#	Stratégie	T	k	p	Notes	Pitches uniques	σ_{vel}	Caractère perçu
1	Création libre	1,0	30	—	217	34	12,3	équilibré, enjoué
2	Conservateur	0,7	15	—	198	15	6,6	calme, fin répétitive
3	Créatif	1,2	—	0,95	235	42	12,6	dissonant, exploratoire
4	Continuation #1	1,0	30	—	212	36	6,4	continuation cohérente
5	Continuation #2	0,9	25	—	211	24	12,2	doux, mélodieux

Table 2 – Caractéristiques des cinq pièces générées. La colonne « pitches uniques » indique combien de hauteurs distinctes sont effectivement utilisées, et σ_{vel} est l’écart-type des vitesses, qu’on peut lire comme une mesure de l’expressivité dynamique.

5.3 Discussion des résultats

Les chiffres confirment ce que l’écoute laisse percevoir. La pièce 2, générée avec une température de 0,7, n’utilise que 15 hauteurs distinctes : le modèle joue presque toujours la note la plus probable, ce qui produit une mélodie cohérente mais qui finit par tourner en rond. La pièce 3, avec une température de 1,2, monte au contraire à 42 hauteurs réparties sur cinq octaves : le modèle prend des risques harmoniques que l’oreille perçoit comme dissonants ou maladroits. C’est entre ces deux extrêmes, autour de $T = 1,0$ et $k = 30$, que la qualité subjective est la meilleure.

Les pièces 4 et 5, générées en continuation d’un *prompt* MIDI réel, illustrent un autre aspect intéressant. La pièce 5 conserve une dispersion en hauteurs très resserrée ($\sigma_{\text{pitch}} = 6,1$, la plus faible du lot), ce qui se traduit par un caractère mélodieux et tonalement stable. Le modèle adapte donc son style à l’amorce qu’on lui donne, sans être paramétré explicitement pour cela.

Un point qui me paraît particulièrement intéressant est que toutes les pièces respectent la « grammaire » REMI, c’est-à-dire l’ordre attendu POSITION ; NOTE_ON ; DURATION ; VELOCITY. Aucune DURATION n’apparaît sans NOTE_ON préalable, et les POSITION ne régressent jamais à l’intérieur d’une mesure. Cette propriété n’est imposée par aucune contrainte explicite : elle émerge simplement du fait que le modèle a appris à prédire correctement le token suivant. C’est un signe que l’apprentissage a saisi non seulement quelles notes sont musicalement plausibles, mais aussi la structure syntaxique de la représentation.

6 Limites et conclusion

Le modèle obtenu présente plusieurs limites assumées. Avec un *context_length* de 256, il perd la trace des structures musicales s’étendant au-delà d’une quinzaine de mesures : on n’observe pas de retour de thèmes ni de forme de type A-B-A. La quantification du temps en seizièmes de *beat* écrase les nuances rythmiques fines comme le *rubato*. La quantification de la vitesse en 32 bins fait perdre une partie des nuances pianistiques. Enfin, le modèle a été entraîné uniquement sur du piano solo : il ne saurait pas générer pour un autre instrument ni gérer plusieurs pistes.

Plusieurs pistes d’amélioration sont envisageables. Augmenter *context_length* jusqu’à 1024 ou plus, en utilisant des *Rotary Position Embeddings* pour limiter la consommation mémoire, permettrait probablement de capturer des structures plus longues. La courbe de perte (figure 1) ne saturait pas à la fin des deux *epochs* : un troisième *epoch* aurait sans doute fait gagner encore quelques dixièmes de perte. Enfin, on pourrait introduire des tokens de contrôle stylistique (genre, tempo, tonalité) pour conditionner explicitement la génération.

Pour conclure, ce projet a permis de vérifier qu’une architecture de transformeur décodeur, telle que

celle présentée en cours pour le texte, se transpose efficacement à la musique symbolique dès lors que la représentation des données est adaptée. La leçon principale que je retiens est que le tokenizer joue un rôle au moins aussi important que le modèle : c'est lui qui définit l'espace dans lequel le modèle apprend. Le passage d'une représentation en *ticks* bruts à une représentation REMI ramène un vocabulaire potentiellement énorme à seulement 157 tokens, et c'est ce qui rend possible un entraînement réussi avec les moyens d'un projet étudiant.